

CURSO PRUEBAS DE SOFTWARE — MÓDULO III

Evidencias de Entrega

API de Categorías con FastAPI

Actividad Autónoma — Construcción y Pruebas de una API REST

Tecnologías: Python · FastAPI · Pydantic · pytest · TestClient

Ficha: 3407180

Este documento contiene la evidencia de ejecución manual (Swagger UI) y automatizada (pytest) de los 12 casos de prueba definidos para la API de Categorías, más 2 casos adicionales del reto opcional de búsqueda por nombre.

1. Documentación interactiva — Swagger UI (/docs)

Vista general generada automáticamente por FastAPI a partir de los modelos Pydantic y las rutas definidas en `app/main.py`. Incluye tanto los endpoints de **Productos** (heredados de la práctica guiada) como los de **Categorías**, objeto de esta actividad autónoma.

default		^
GET	/ Root	▼
GET	/products/ Get Product	▼
GET	/products/{product_id} Get Product By Id	▼
PATCH	/products/{product_id} Update Product	▼
DELETE	/products/{product_id} Delete Product	▼
POST	/products Create Product	▼
GET	/health Health Check	▼
GET	/categories Get Categories	▼
POST	/categories Create Category	▼
GET	/categories/{category_id} Get Category	▼
PATCH	/categories/{category_id} Update Category	▼
DELETE	/categories/{category_id} Delete Category	▼

Figura 1. Listado completo de endpoints disponibles en /docs.

2. Evidencia de pruebas manuales en Swagger

Cada caso se probó manualmente desde Swagger UI, con captura de la petición y la respuesta del servidor. Cuando la propia interfaz de Swagger valida el dato antes de enviarlo (como ocurre con id no numéricos), se incluye además la evidencia de la petición real vista desde la terminal del servidor.

GET /categories — Listar categorías

Resultado esperado: 200 y lista JSON

Se listan las categorías registradas en memoria.

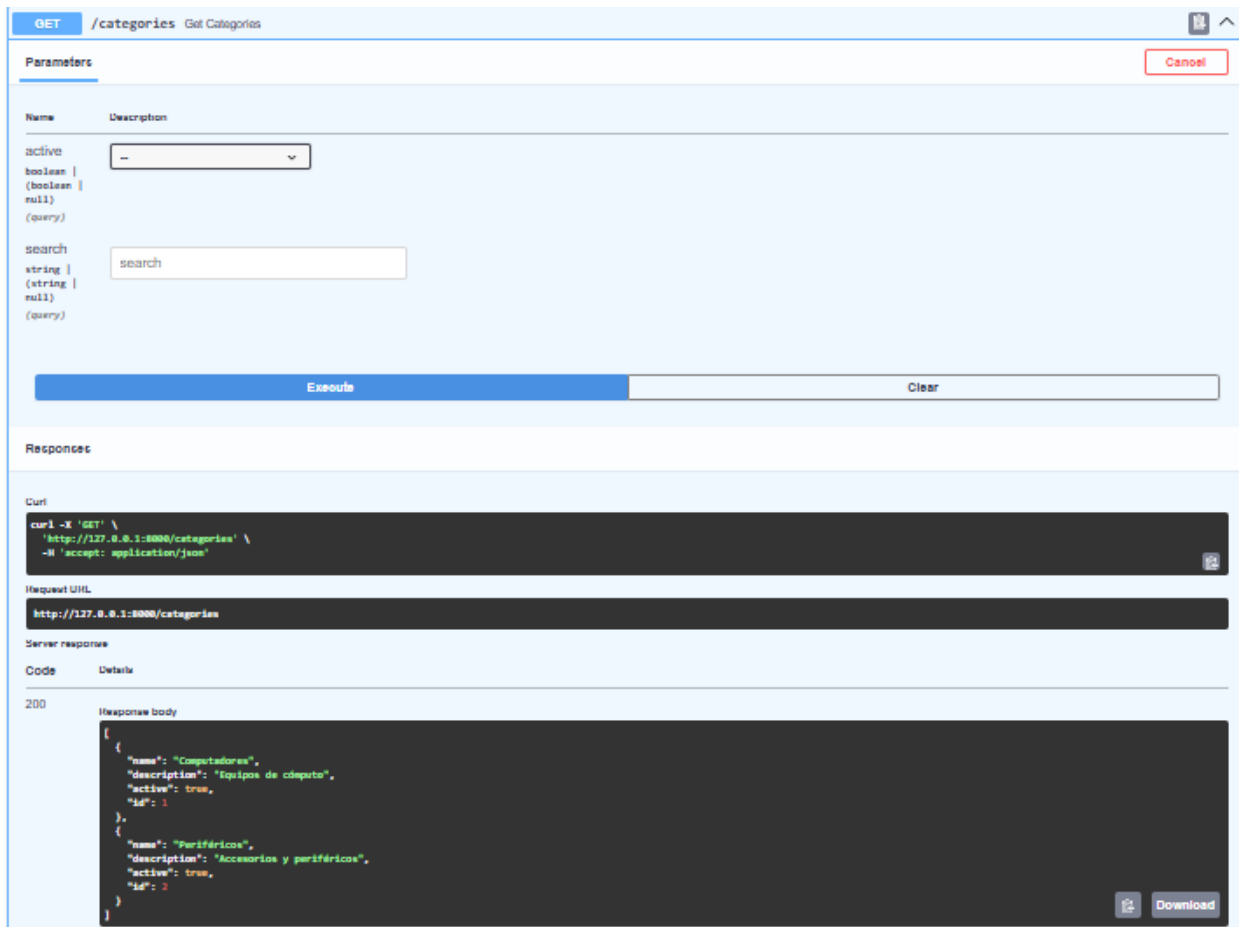


Figura 2 (CA01). Respuesta 200 con el arreglo completo de categorías.

GET

/categories/{category_id} — Consultar categoría existente

Resultado esperado: 200

Se consulta la categoría con id=2 y se obtiene su detalle completo.

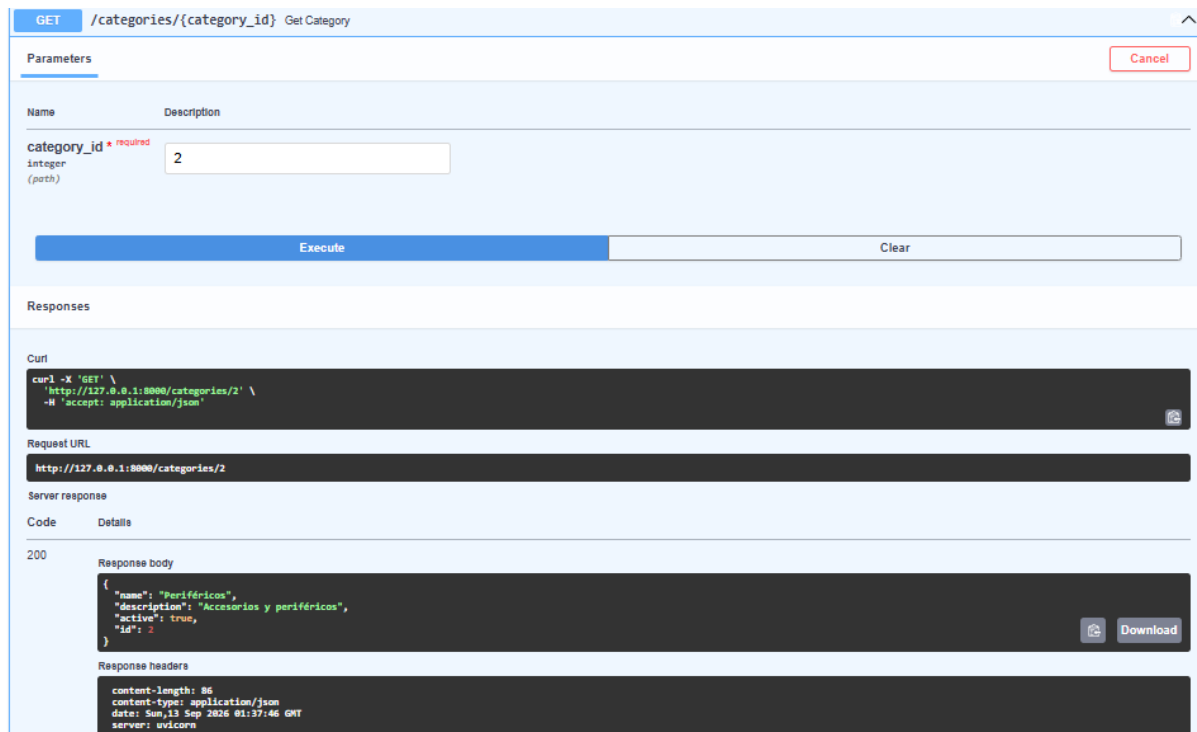


Figura 3 (CA02). Respuesta 200 con los datos de la categoría 'Periféricos'.

GET

/categories/{category_id} — Consultar categoría inexistente

Resultado esperado: 404

Se consulta un id que no existe (56) y la API responde con el mensaje de error esperado.

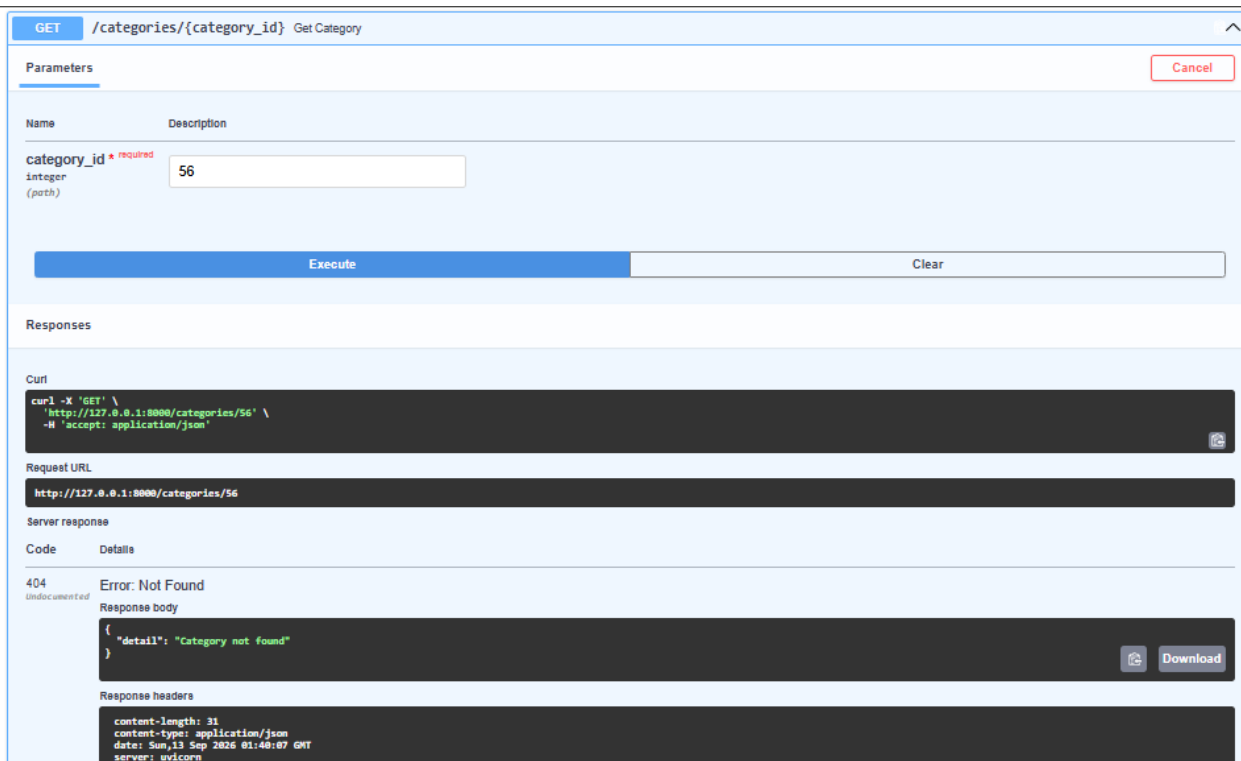


Figura 4 (CA03). Respuesta 404 con detail: 'Category not found'.

GET /categories/{category_id} — ID inválido (no numérico)

Resultado esperado: 422

Se envía un valor no numérico como id.

The image shows the Swagger UI interface for the endpoint `GET /categories/{category_id}`. The parameter `category_id` is defined as an integer (path). A red box highlights the validation error: "Please correct the following validation errors and try again." with the message "For 'category_id': Value must be an integer." The input field contains the text "nam". The "Execute" button is disabled, and the "Clear" button is visible.

Figura 5a (CA04). Swagger UI bloquea el envío en el navegador y muestra su propio cartel de validación ('Value must be an integer').

```
PS D:\Modulo_3_SANTIAGO> python -m uvicorn app.main:app --reload
INFO: waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process [27244]
INFO: Started server process [16628]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:56005 - "GET /categories/abc HTTP/1.1" 422 Unprocessable Content
```

Figura 5b (CA04). Petición real GET /categories/abc hecha directamente (fuera de Swagger); el log de uvicorn confirma el 422 generado por el servidor.

Nota: El botón *Execute* de Swagger valida el tipo de dato en el propio navegador antes de enviar la petición, por lo que esa captura por sí sola no demuestra el comportamiento del servidor. Por eso se incluye una segunda evidencia (Figura 5b) con la petición real, que sí llega a la API y confirma el 422 en el log de uvicorn.

POST /categories — Crear categoría válida

Resultado esperado: 201

Se crea la categoría 'raton' con datos válidos.

POST /categories Create Category

Parameters

No parameters

Request body *required* application/json

Edit Value | Schema

```
{
  "name": "raton",
  "description": "Equipos de cómputo",
  "active": true
}
```

Execute Clear

Responses

Figura 6a (CA05). Cuerpo de la petición POST.

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/categories' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "raton",
    "description": "Equipos de cómputo",
    "active": true
  }'
```

Request URL

http://127.0.0.1:8000/categories

Server response

Code	Details
201	Response body <pre>{ "name": "raton", "description": "Equipos de cómputo", "active": true, "id": 3 }</pre> Response headers <pre>content-length: 73 content-type: application/json date: Sun, 13 Sep 2026 02:23:06 GMT server: uvicorn</pre>

Responses

Code	Description	Links
201	Successful Response	No links

Figura 6b (CA05). Respuesta 201 con el id asignado por el servidor (id: 3).

```
INFO: Application startup complete.
INFO: 127.0.0.1:56005 - "GET /categories/abc HTTP/1.1" 422 Unprocessable Content
INFO: 127.0.0.1:51217 - "POST /categories HTTP/1.1" 201 Created
```

Figura 6c (CA05). Log de uvicorn confirmando 201 Created.

POST /categories — Nombre demasiado corto

Resultado esperado: 422

Se envía name='ra' (2 caracteres), por debajo del mínimo de 3 exigido por el modelo.

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Raw Value | Schema

```
{
  "name": "ra",
  "description": "Equipos de cómputo",
  "active": true
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  http://127.0.0.1:8000/categories/ \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "ra",
    "description": "Equipos de cómputo",
    "active": true
  }'
```

Request URL

http://127.0.0.1:8000/categories

Server response

Code	Details
422	Error: Unprocessable Content Response body <pre>{ "detail": { "type": "string_too_short", "loc": { "body", "name" } } }</pre>

Figura 7a (CA06). Respuesta 422 con detail tipo 'string_too_short'.

```
INFO: 127.0.0.1:56005 - "GET /categories/abc HTTP/1.1" 422 Unprocessable Content
INFO: 127.0.0.1:51217 - "POST /categories HTTP/1.1" 201 Created
INFO: 127.0.0.1:61630 - "POST /categories HTTP/1.1" 201 Created
INFO: 127.0.0.1:63464 - "POST /categories HTTP/1.1" 422 Unprocessable Content
```

Figura 7b (CA06). Log de uvicorn confirmando 422 Unprocessable Content.

POST

/categories — Falta el nombre

Resultado esperado: 422

Se envía name="" (cadena vacía), que tampoco cumple el mínimo de 3 caracteres.

POST /categories Create Category

Parameters

No parameters

Request body *required* application/json

body Value | Schema

```
{
  "name": "",
  "description": "Equipo de cómputo",
  "active": true
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/categories' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "",
    "description": "Equipo de cómputo",
    "active": true
  }'
```

Request URL

```
http://127.0.0.1:8000/categories
```

Server response

Code	Detail
422	Error: Unprocessable Content

Response body

```
{
  "detail": {
    "type": "string_too_short",
    "msg": "..."
  }
}
```

Figura 8a (CA07). Respuesta 422 con detail tipo 'string_too_short'.

```
INFO: 127.0.0.1:63464 - "POST /categories HTTP/1.1" 422 Unprocessable Content
INFO: 127.0.0.1:60747 - "POST /categories HTTP/1.1" 422 Unprocessable Content
```

Figura 8b (CA07). Log de uvicorn confirmando 422 Unprocessable Content.

PATCH /categories/{category_id} — Actualizar categoría existente

Resultado esperado: 200

Se actualiza la categoría id=1, cambiando su nombre a 'Portátil gamer' y su descripción.

The screenshot shows a REST client interface with the following details:

- URL:** `/categories/{category_id}` with the method `Update Category`.
- Parameters:** A single parameter `category_id` of type `integer (path)` with the value `1`.
- Request body:** A JSON object: `{ "name": "Perkatis game", "description": "Epique perikatis", "active": true }`. The content type is set to `application/json`.
- Response:** A `200` status code with a JSON body: `{ "name": "Perkatis game", "description": "Epique perikatis", "active": true, "id": 1 }`. The response headers show a content length of 82, content type of `application/json`, and a date of `Sun, 11 Sep 2022 05:12:18 GMT`.

Figura 9 (CA08). Respuesta 200 con los campos actualizados.

■ **Observación:** Esta actualización renombra la categoría que originalmente se llamaba 'Computadores'. Esto tiene un efecto colateral importante que se explica en la sección de Observaciones (afecta el resultado de la búsqueda por nombre más adelante en esta misma sesión manual).

PATCH /categories/{category_id} — Actualizar categoría inexistente

Resultado esperado: 404

Se intenta actualizar el `id=99`, que no existe.

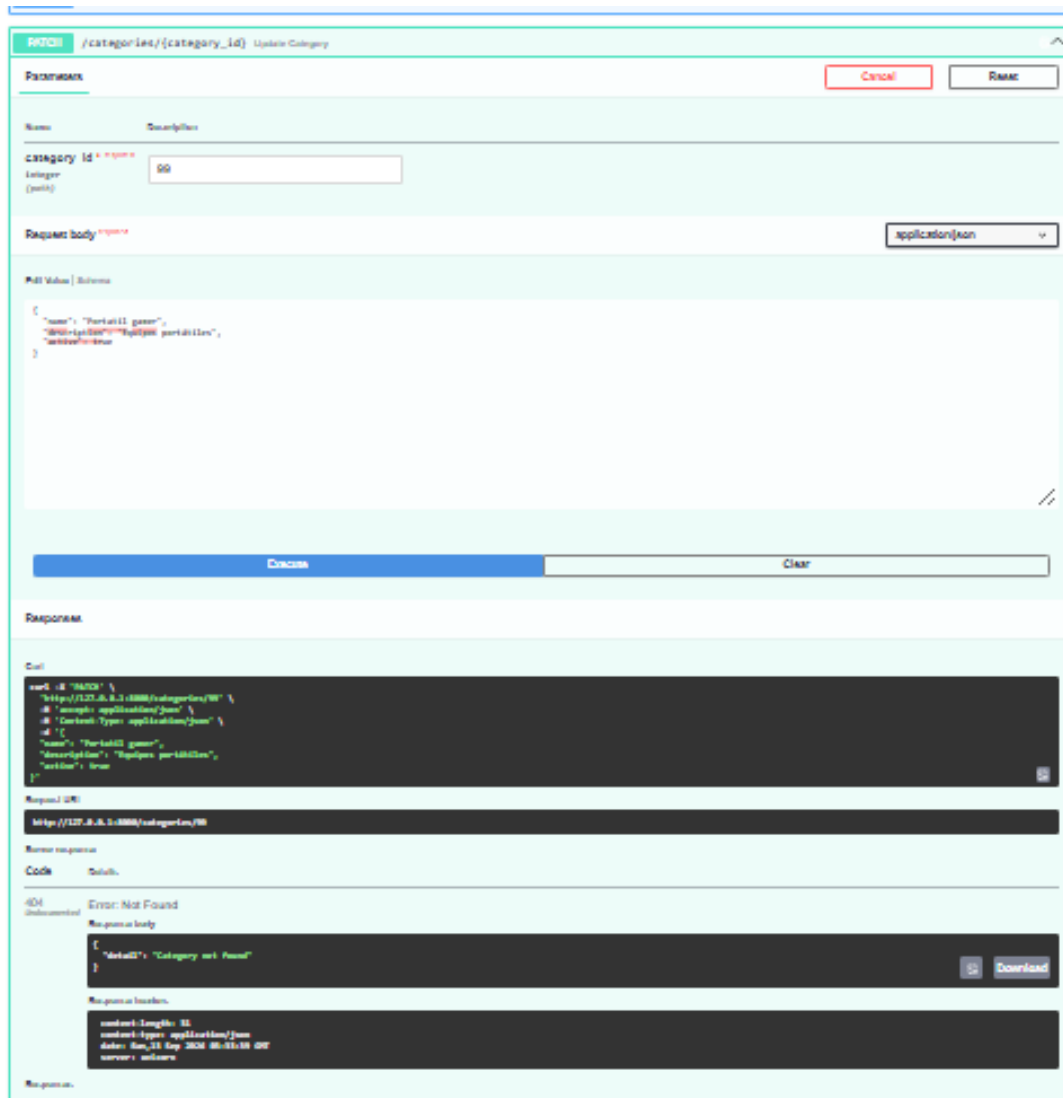


Figura 10 (CA09). Respuesta 404 con detail: 'Category not found'.

DELETE

`/categories/{category_id}` — Eliminar categoría existente

Resultado esperado: 204

Se elimina la categoría id=4 (creada previamente durante la sesión de pruebas manuales).

DELETE /categories/{category_id} Delete Category

Parameters

Name	Description
category_id required	
integer	
(path)	

4

ExecuteClear

Responses

Curl

```
curl -X 'DELETE' \
  'http://127.0.0.1:8000/categories/4' \
  -H 'accept: */*'
```

Request URL

```
http://127.0.0.1:8000/categories/4
```

Server response

Code	Details
204	Response headers <pre>date: Sun, 13 Sep 2020 03:35:16 GMT server: uvicorn</pre>

Responses

Code	Description	Links
204	Successful Response	No links
422	Validation Error	No links

Media type

application/json

Example Value | Schema

Figura 11 (CA10). Respuesta 204 sin cuerpo, tal como exige el estándar HTTP.

DELETE

/categories/{category_id} — Eliminar categoría inexistente

Resultado esperado: 404

Se intenta eliminar el id=10000, que no existe.

API de Categorías con FastAPI — Evidencias de Entrega

Página 11

DELETE

/categories/{category_id} Delete Category

Cancel

Parameters

Name	Description
category_id * required	
integer	10000
(path)	

ExecuteClear

Responses

Curl

```
curl -X 'DELETE' \
  'http://127.0.0.1:8000/categories/10000' \
  -H 'accept: */*'
```

Request URL

```
http://127.0.0.1:8000/categories/10000
```

Server response

Code	Details
404	Error: Not Found

Undocumented

Response body

```
{
  "detail": "Category not found"
}
```

Download

Response headers

```
content-length: 33
content-type: application/json
date: Sun, 13 Sep 2020 03:36:16 GMT
server: uvicorn
```

Figura 12 (CA11). Respuesta 404 con detail: 'Category not found'.

3. Pruebas manuales adicionales: filtros y búsqueda

Durante la sesión manual también se probaron el filtro `search` (reto opcional) y el filtro `active`. A diferencia de la suite automatizada, estas pruebas manuales se ejecutaron en secuencia sobre la **misma base de datos en memoria**, sin reiniciarla entre una prueba y otra — por eso sus resultados no deben leerse de forma aislada.

GET

`/categories?search=computadores` — Reto opcional: búsqueda por nombre

Resultado esperado: 200

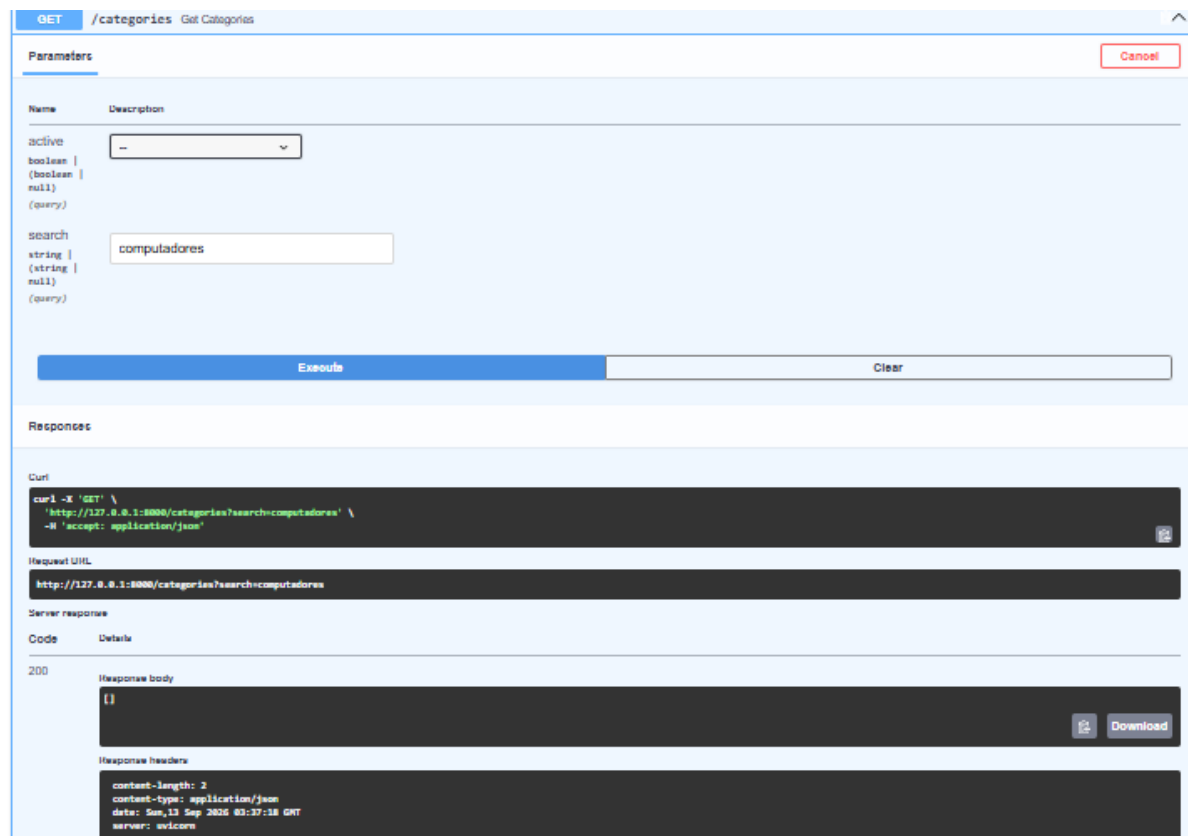


Figura 13. La búsqueda de 'computadores' devuelve una lista vacía.

■ **Observación:** La lista vacía es correcta dado el estado acumulado de la sesión: en el CA08 (Figura 9) la categoría que se llamaba 'Computadores' fue renombrada a 'Portátil gamer'. Para el momento de esta prueba, ninguna categoría contiene la palabra 'computadores' en su nombre, así que `[]` es la respuesta correcta — no indica un error de la búsqueda. La suite automatizada (sección 4) sí valida este caso de forma aislada y confirma que la búsqueda encuentra resultados cuando corresponde.

GET

`/categories?active=false` — Filtro por estado (inactivas)

Resultado esperado: 200

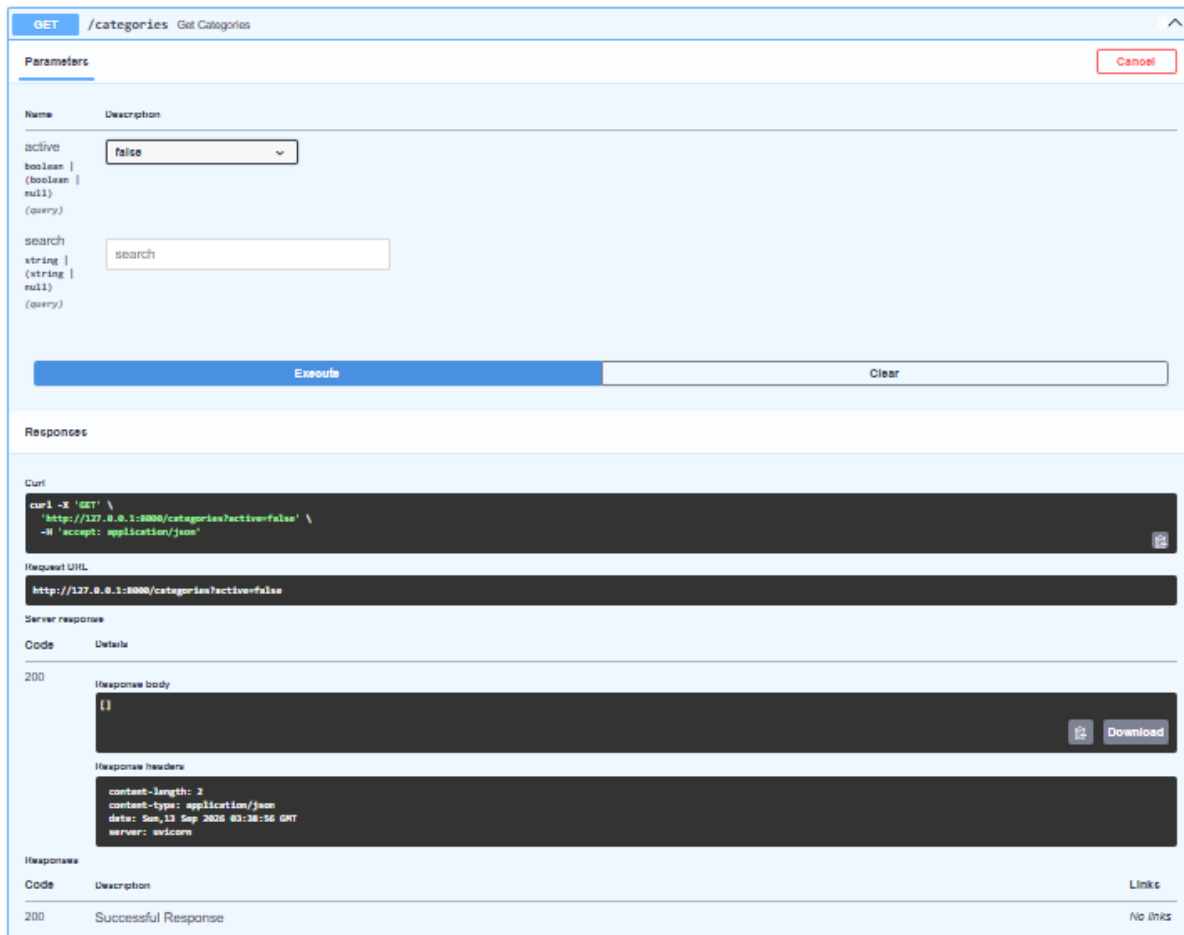


Figura 14. El filtro `active=false` devuelve una lista vacía.

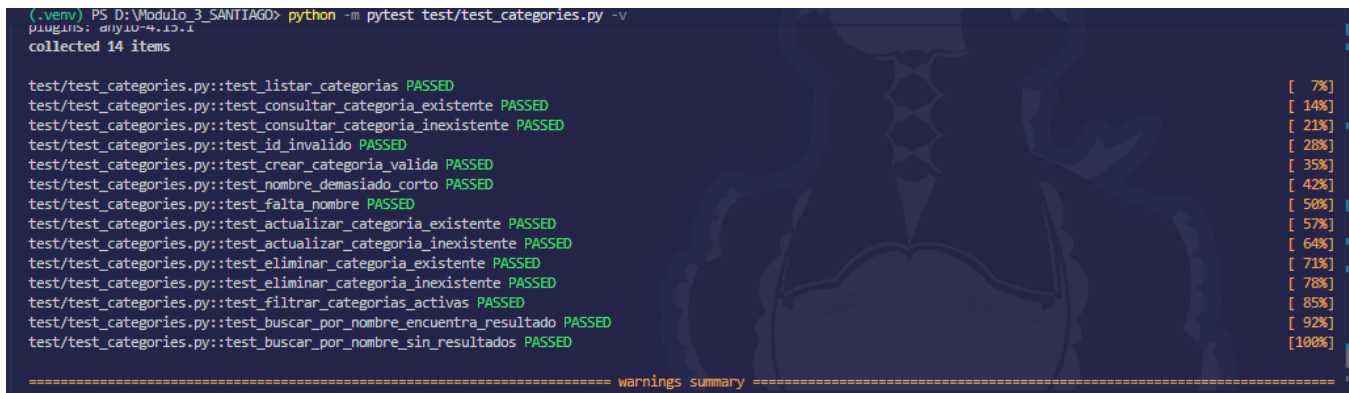
■ **Observación:** Para este punto de la sesión manual, todas las categorías presentes en la base de datos en memoria tienen `active=true` (ninguna se creó o actualizó como inactiva durante la secuencia de pruebas), por lo que `[]` es el resultado correcto. La condición del PDF (CA12) pide específicamente probar `?active=true`; se recomienda repetir esta captura con ese valor para dejar evidencia exacta del caso de la matriz. El caso sí está cubierto y pasa correctamente en la suite automatizada de `pytest`.

4. Ejecución de pruebas automatizadas (pytest)

A diferencia de las pruebas manuales en Swagger, la suite automatizada usa `fastapi.testclient.TestClient` junto con un fixture `autouse=True` que restablece `categories_db` antes de cada prueba. Esto garantiza que cada caso se ejecute siempre sobre el mismo estado inicial, sin los efectos colaterales observados en la sección 3.

Comando ejecutado:

```
python -m pytest test/test_categories.py -v
```



```
(.venv) PS D:\Modulo_3_SANTIAGO> python -m pytest test/test_categories.py -v
plugins: dirtyio-4.13.1
collected 14 items

test/test_categories.py::test_listar_categorias PASSED [ 7%]
test/test_categories.py::test_consultar_categoria_existente PASSED [ 14%]
test/test_categories.py::test_consultar_categoria_inexistente PASSED [ 21%]
test/test_categories.py::test_id_invalido PASSED [ 28%]
test/test_categories.py::test_crear_categoria_valida PASSED [ 35%]
test/test_categories.py::test_nombre_demasiado_corto PASSED [ 42%]
test/test_categories.py::test_falta_nombre PASSED [ 50%]
test/test_categories.py::test_actualizar_categoria_existente PASSED [ 57%]
test/test_categories.py::test_actualizar_categoria_inexistente PASSED [ 64%]
test/test_categories.py::test_eliminar_categoria_existente PASSED [ 71%]
test/test_categories.py::test_eliminar_categoria_inexistente PASSED [ 78%]
test/test_categories.py::test_filtrar_categorias_activas PASSED [ 85%]
test/test_categories.py::test_buscar_por_nombre_encuentra_resultado PASSED [ 92%]
test/test_categories.py::test_buscar_por_nombre_sin_resultados PASSED [100%]

===== warnings summary =====
```

Figura 15. Resultado real de la ejecución: **14 passed** — las 12 pruebas obligatorias de la matriz (CA01–CA12) más 2 pruebas del reto opcional de búsqueda.

5. Matriz de casos de prueba — resultado obtenido

Resultado consolidado de cada caso, tomando como fuente de verdad la ejecución automatizada con pytest (sección 4), que aísla cada prueba y no depende del orden de ejecución.

ID	Escenario	Petición / condición	Esperado	Resultado
CA01	Listar categorías	GET /categories	200 y lista JSON	PASS
CA02	Consultar existente	GET /categories/2	200	PASS
CA03	Consultar inexistente	GET /categories/56	404	PASS
CA04	ID inválido	GET /categories/abc	422	PASS
CA05	Crear válida	POST con datos correctos	201	PASS
CA06	Nombre demasiado corto	POST con name de 1-2 caracteres	422	PASS
CA07	Falta nombre	POST sin name	422	PASS
CA08	Actualizar existente	PATCH /categories/1	200	PASS
CA09	Actualizar inexistente	PATCH /categories/999	404	PASS
CA10	Eliminar existente	DELETE /categories/1	204	PASS
CA11	Eliminar inexistente	DELETE /categories/999	404	PASS
CA12	Filtrar activas	GET /categories?active=true	200, solo active=true	PASS
Extra 1	Reto opcional: búsqueda con resultado	GET /categories?search=comp	200, con coincidencias	PASS
Extra 2	Reto opcional: búsqueda sin resultado	GET /categories?search=xyz	200, lista vacía	PASS

14 / 14 casos en PASS. Las evidencias manuales de las secciones 2 y 3 respaldan visualmente estos resultados; donde una captura manual mostró un resultado aparentemente distinto (Figuras 13 y 14), se documentó la causa exacta y se confirmó que el caso correspondiente sí pasa en la suite automatizada.

6. Observaciones adicionales

Swagger valida en el navegador, no en el servidor

Para campos definidos como `integer` en la ruta (como `category_id`), Swagger UI revisa el tipo de dato con JavaScript antes de dar clic en 'Execute'. Si el valor no es numérico, bloquea el envío y muestra su propio cartel de error — la petición nunca llega al servidor y por eso no aparece nada en la terminal de `uvicorn`. Para verificar el 422 real generado por FastAPI/Pydantic hay que hacer la petición fuera de Swagger: escribiendo la URL directamente en el navegador, con `curl`, o dejando que la valide `pytest`.

Las pruebas manuales comparten estado; las automatizadas no

Al probar manualmente en Swagger, cada petición modifica la misma lista `categories_db` en memoria, y esos cambios persisten para las siguientes pruebas de la sesión. Esto explica por qué la búsqueda por 'computadores' y el filtro por inactivas devolvieron listas vacías (Figuras 13 y 14): categorías anteriores ya habían sido renombradas o nunca se creó ninguna inactiva. La suite de `pytest` evita este problema con un fixture que reinicia la base de datos antes de cada prueba, garantizando resultados independientes del orden de ejecución — tal como exige el punto 10 del PDF de la actividad.

Captura de CA12 no coincide exactamente con el enunciado

El PDF pide evidenciar `GET /categories?active=true`, pero la captura disponible corresponde a `active=false`. Ambas usan la misma lógica de filtrado y el caso sí pasa en `pytest`, pero se recomienda repetir la captura manual con `active=true` para que la evidencia visual coincida exactamente con la matriz de casos entregada.

El id generado no es secuencial estricto entre pruebas manuales

Durante la sesión manual se crearon y eliminaron varias categorías (algunas rechazadas por validación), por lo que los ids que aparecen en las capturas (3, 4, etc.) no necesariamente coinciden con los que obtendría alguien repitiendo la secuencia exacta. Esto es un comportamiento esperado de una base de datos en memoria con ids autoincrementales, no un error de la API.

Documento generado a partir de las capturas originales de Swagger UI y de la ejecución real de la suite de `pytest` del proyecto. Curso Pruebas de Software — Módulo III.